



Certified Kubernetes Administrator (CKA)

By Cloud Mentor Pro

Updated: 10/04/2026



Mock Exam - 2 (Level Up!)

- *8 Questions*

Q1

1 2 3 4 5 6 7 8

Weight: 10

Take a backup of the etcd cluster and save it to `/opt/etcd-backup.db`.

● Backup Completed

Q1 - Answer

```
controlplane ~ → ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \  
> --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key \  
> snapshot save /opt/etcd-backup.db  
Snapshot saved at /opt/etcd-backup.db  
  
controlplane ~ → ls  
CKA etcd-v3.3.1-linux-amd64 etcd-v3.3.1-linux-amd64.tar.gz sample.yaml  
  
controlplane ~ → ls /opt  
cni containerd etcd-backup.db  
  
controlplane ~ → ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/etcd-backup.db  
+-----+-----+-----+-----+  
| HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |  
+-----+-----+-----+-----+  
| 1fc57afb | 5161 | 843 | 2.2 MB |  
+-----+-----+-----+-----+  
  
controlplane ~ →
```

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt  
--key=/etc/kubernetes/pki/etcd/server.key \  
snapshot save /opt/etcd-backup.db
```

#Verify

```
ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/etcd-backup.db
```

Q2



1 2 3 4 5 6 7 8



Weight: 10

Create a Pod called `redis-storage` with image: `redis:alpine` with a Volume of type `emptyDir` that lasts for the life of the Pod.

Specs on the below.

- Pod named 'redis-storage' created
- Pod 'redis-storage' uses Volume type of emptyDir
- Pod 'redis-storage' uses volumeMount with mountPath = /data/redis

Q2 - Answer

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: redis-storage
  name: redis-storage
spec:
  volumes:
  - name: share-data
    emptyDir: {}
  containers:
  - image: redis:alpine
    name: redis-storage
    resources: {}
    volumeMounts:
    - mountPath: /data/redis
      name: share-data
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

```
Containers:
  redis-storage:
    Container ID:   containerd://77e8937a6bcb5740da14b88c2b44cac39e98449b72691119dad50369900b6e20
    Image:          redis:alpine
    Image ID:       docker.io/library/redis@sha256:a3e82d34456f5d012dcbf542b564c39697c14dc4225cf3d29b3278fca041ad15
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Sat, 09 Dec 2023 21:47:56 -0500
    Ready:         True
    Restart Count:  0
    Environment:   <none>
    Mounts:
      /data/redis from share-data (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-8qb5w (ro)
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  share-data:
    Type:          EmptyDir (a temporary directory that shares a pod's lifetime)
    Medium:
    SizeLimit:     <unset>
```

```
controlplane ~ → k get po
NAME          READY   STATUS    RESTARTS   AGE
redis-storage 1/1     Running   0           2m56s
```

Q2 - Answer

```
k run redis-storage --image=redis:alpine --dry-run=client -o yaml > 2.yaml
```

```
vi 2.yaml #Edit them volumes & volumeMounts
```

```
spec:
```

```
  volumes:
```

```
  - name: share-data
```

```
    emptyDir:
```

```
  containers:
```

```
  - image: redis:alpine
```

```
    name: redis-storage
```

```
    resources: {}
```

```
    volumeMounts:
```

```
    - mountPath: /data/redis
```

```
      name: share-data
```

```
#Verify
```

```
k exec -it redis-storage -- /bin/sh
```

```
ls /data/redis/
```

Q3



1 2 **3** 4 5 6 7 8



Weight: 8

Create a new pod called `super-user-pod` with image `busybox:1.28`. Allow the pod to be able to set `system_time`.

The container should sleep for 4800 seconds.

- Pod: super-user-pod
- Container Image: busybox:1.28
- Is SYS_TIME capability set for the container?

Q3 - Answer

```
k run super-user-pod --image=busybox:1.28 --command sleep 4800 --dry-run=client -o
```

```
yaml > 3.yaml
```

```
vi 3.yaml #Them capabilities for container
```

```
spec:
```

```
  containers:
```

```
    - command:
```

```
      - sleep
```

```
      - "4800"
```

```
    image: busybox:1.28
```

```
    name: super-user-pod
```

```
    securityContext:
```

```
      capabilities:
```

```
        add: ["SYS_TIME"]
```

Q4



1 2 3 4 5 6 7 8



Weight: 12

A pod definition file is created at `/root/CKA/use-pv.yaml`. Make use of this manifest file and mount the persistent volume called `pv-1`. Ensure the pod is running and the PV is bound.

mountPath: `/data`

persistentVolumeClaim Name: `my-pvc`

- ☐ persistentVolume Claim configured correctly
- ☐ pod using the correct mountPath
- ☐ pod using the persistent volume claim?

Q4 - Answer

```
controlplane ~ ✦✧ vi 4_pvc.yaml
```

```
controlplane ~ ✦✧➔ k get pv,pvc
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	REASON	AGE
persistentvolume/pv-1	10Mi	RWO	Retain	Bound	default/my-pvc			4m36s

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
persistentvolumeclaim/my-pvc	Bound	pv-1	10Mi	RWO		4s

```
controlplane ~ ✦✧➔
```

Mounts:

/data from my-data (rw)

/var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-78xpf (ro)

Conditions:

Type	Status
Initialized	True
Ready	True
ContainersReady	True
PodScheduled	True

Volumes:

my-data:

Type: PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)

ClaimName: my-pvc

ReadOnly: false

Q4 - Answer

Step1: Create a new pvc

```
k get pv,pvc #Khong thay pvc => Can tao pvc  
vi 4_pvc.yaml
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
```

```
metadata:
```

```
  name: my-pvc
```

```
spec:
```

```
  accessModes:
```

```
    - ReadWriteOnce
```

```
  resources:
```

```
    requests:
```

```
      storage: 10Mi
```

Step2: Correct Pod.yaml

Q5



1 2 3 4 5 6 7 8



Weight: 15

Create a new deployment called `nginx-deploy`, with image `nginx:1.16` and `1` replica. Next upgrade the deployment to version `1.17` using rolling update.

- Deployment : nginx-deploy. Image: nginx:1.16
- Image: nginx:1.16
- Task: Upgrade the version of the deployment to 1:17
- Task: Record the changes for the image upgrade

Q5 - Answer

```
controlplane ~ → k apply -f 5.yaml
deployment.apps/nginx-deploy configured

controlplane ~ → k describe deployments.apps nginx-deploy
Name:                nginx-deploy
Namespace:           default
CreationTimestamp:    Sun, 10 Dec 2023 07:29:37 -0500
Labels:              app=nginx-deploy
Annotations:         deployment.kubernetes.io/revision: 2
Selector:             app=nginx-deploy
Replicas:            1 desired | 1 updated | 2 total | 1 available | 1 unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-deploy
  Containers:
    nginx:
      Image:      nginx:1.17
      Port:       <none>
      Host Port:   <none>
      Environment: <none>
      Mounts:      <none>
      Volumes:     <none>
  Conditions:
    Type           Status  Reason
    ----           -
    Available      True    MinimumReplicasAvailable
    Progressing    True    ReplicaSetUpdated
OldReplicaSets:  nginx-deploy-dcbd487f9 (1/1 replicas created)
NewReplicaSet:   nginx-deploy-5c95467974 (1/1 replicas created)
Events:
  Type    Reason             Age    From                  Message
  ----    -
  Normal  ScalingReplicaSet  62s    deployment-controller  Scaled up replica set nginx-deploy-dcbd487f9 to 1
  Normal  ScalingReplicaSet  2s     deployment-controller  Scaled up replica set nginx-deploy-5c95467974 to 1
```

k create deployment nginx-deploy --image nginx:1.16 --replicas=1 --dry-run=client -o yaml > 5.yaml

k apply -f 5.yaml

vi 5.yaml #Change image from "nginx:1.16" to "nginx:1.17"

Q6



1 2 3 4 5 **6** 7 8



Weight: 15

Create a new user called `john` . Grant him access to the cluster. John should have permission to `create, list, get, update and delete pods` in the `development` namespace . The private key exists in the location: `/root/CKA/john.key` and csr at `/root/CKA/john.csr` .

Important Note : As of kubernetes 1.19, the CertificateSigningRequest object expects a `signerName` .

Please refer the documentation to see an example. The documentation tab is available at the top right of terminal.

- CSR: john-developer Status:Approved
- Role Name: developer, namespace: development, Resource: Pods
- Access: User 'john' has appropriate permissions

Q6 - Answer

[illegible]

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: john-developer
spec:
  request: LS0tLS1CRUdJTiBDRVRUSUJZQ0FURSB5RVFRVWVNUStLS0KU0t1UjQ1ZEQ0NBVHdQVFBd0RGRUSXNQXNHQTFVRUF3d0VhbTlVYmpDQ0TSXDEUUVKS29aSWh2Y0S0
  UUVQClFBRApn20VQQUdRQ0FRb0NnZ0V0QUt5WHF2L2x0x0Wk4V29kL1ZGVL1VURTU1KNTZhMlUza2NweC9YeJb0RvH0tEVCNFCXClRrWnhxwE8xVzBtTh1nTExtEhU4RndBZ1Vwlbm5
  pSC9SB1F1ZHf3R0g15kZpVUWHQxc1RmhRZU1JNERkWGpQMTIKRUNKEmRYUwpm09sLzRYTE01dHNXcGt4UUVjREhSb3c3cxPbBjJEaTJSRmUza2STSEN0cFFpM21RSTBvNkr3Ba
  pZUD1KVnhHmE4WgEjQXZwaT1lM3KZi4t4ZGNoZnJjdKdkVFhJd2RINXZ3eno3NTHB0Xp3RDZ2CjBvOW43VjdZCjJzeE02NjE1bjRlIakFwQXZUa3pKVfBSKzFRSUTwakREaGNFR
  W5URWRF05FdTQWduRtNkQU135TVTFmgzT8jaefKGNu94SHgwRTdJK2pRZzTtTkxbG4YnBgRG34OfdndnBGeTY4TEjJ0QF3RUFBYUfBTUEwR0NTcldtSWIzRFFQgPd1VBQTRJ
  KtFRQ0kwwf1FTU1SNudrd1VvbmlBS9hqNVVWVm93ZmVUcVpU0Uc5QnVzS3HheIROcDFC0tMwNFRECK1hL2RESGJCcnJGNXNRTWFZU1ZZZ1kQzRsVwIUeF2Qk14Y21wOE2UnJ
  QDN6ZFRnaGfJ20f40UF51lqMkoKsZArVv1VMDRzbznjb1Ug1L2k4amNt61HawB8VnNpRU2o3d2j1VXhQMxTY2t4SFQ1VkoMNUU1cd1pS0m0Wl2lyZAp1bEpFQM1vc1I20G1Ia1
  FuFmFmUUVNZ0yUmfXUHNNEG82MgPqem3uSmt1b0p3DEJ5Y1dSDDWRT1IcmRxbUJ3CnMwNpnlk84d1A1aFNNODf1b1pSL3BLT29Yck43S5RzmVFOFntcjEvNGZ0a3hFbjJLb
  2pKtEXFaXNDZNRmW0kVwXN21LN2Zk3c1SGdMwN0R3JGRkntWpETTV5QmUKLS0tLS1FtkQ0gVSVE1GSUNBVEUgUkVRVUVTVC0tLS0tCg==
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400 # one day
  usages:
    - client auth
```

```

controlplane ~ → k get crs
NAME      AGE    SIGNATURE
ztn       21m    kubernet...
csr-8696   20m    kubernet...
john-developer 13s    kubernet...

controlplane ~ → kubectl create role developer --verb=create,list,get,update,delete --resource=pods -n development -o yaml > 6_role.yaml

controlplane ~ → k apply -f 6_role.yaml
Warning: resource roles/developer is missing the kubect.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
role.rbac.authorization.k8s.io/developer configured

controlplane ~ → k describe role developer -n development
Name:         developer
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources            Non-Resource URLs   Resource Names       Verbs
  -----
  pods                 []                  []                   [create list get update delete]

controlplane ~ → k create rolebinding developer-role-binding --user=john --role=developer -n development -o yaml > 6_role-binding.yaml
controlplane ~ → k apply -f 6_role-binding.yaml
Warning: resource rolebindings/developer-role-binding is missing the kubect.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
rolebinding.rbac.authorization.k8s.io/developer-role-binding configured

controlplane ~ → k describe rolebindings.rbac.authorization.k8s.io developer-role-binding
Error from server (Notfound): rolebindings.rbac.authorization.k8s.io "developer-role-binding" not found

controlplane ~ → k describe rolebindings.rbac.authorization.k8s.io developer-role-binding -n development
Name:         developer-role-binding
Labels:       <none>
Annotations:  <none>
Role:
  Kind: Role
  Name: developer
Subjects:
  Kind  Name  Namespace
  ----  ---  -
  User  john

controlplane ~ → k auth can-i create pods -n development --as=john
no

controlplane ~ → k auth can-i create deployment -n development --as=john
no

controlplane ~ → k auth can-i get pods -n development --as=john
yes

controlplane ~ → k auth can-i create pods -n development --as=john
yes

```


Q6 - Answer

```
cat /root/CKA/john.csr | base64 | tr -d "\n"
vi 6_csr.yaml #Copy template csr from k8s docs
ka 6_csr.yaml
k get csr
k certificate approve john-developer
```

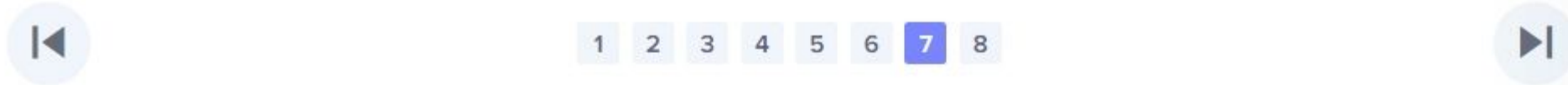
```
kubectl create role developer --verb=create,list,get,update,delete --resource=pods -n development -o
yaml > 6_role.yaml
ka 6_role.yaml
```

```
k create rolebinding developer-role-binding --user=john --role=developer -n development -o yaml >
6_role-binding.yaml
ka 6_role-binding.yaml
```

#Verify

```
k auth can-i create pods -n development --as=join
```

Q7



Weight: 15

Create a nginx pod called `nginx-resolver` using image `nginx`, expose it internally with a service called `nginx-resolver-service`. Test that you are able to look up the service and pod names from within the cluster. Use the image: `busybox:1.28` for dns lookup. Record results in `/root/CKA/nginx.svc` and `/root/CKA/nginx.pod`

- Pod: nginx-resolver created
- Service DNS Resolution recorded correctly
- Pod DNS resolution recorded correctly

Q7 - Answer

```
controlplane ~ → kubectl run nginx-resolver --image=nginx
pod/nginx-resolver created

controlplane ~ → kubectl expose pod nginx-resolver --name=nginx-resolver-service --port=80 --target-port=80 --type=ClusterIP
service/nginx-resolver-service exposed

controlplane ~ → kubectl run test-nslookup --image=busybox:1.28 -- sleep 1d
pod/test-nslookup created

controlplane ~ → k get pod
NAME          READY   STATUS    RESTARTS   AGE
nginx-resolver 1/1     Running   0           33s
test-nslookup  0/1     ContainerCreating 0           3s

controlplane ~ → k get pod
NAME          READY   STATUS    RESTARTS   AGE
nginx-resolver 1/1     Running   0           42s
test-nslookup 1/1     Running   0           12s

controlplane ~ → k exec test-nslookup -it -- nslookup nginx-resolver-service
Server:      10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      nginx-resolver-service
Address 1: 10.100.159.90 nginx-resolver-service.default.svc.cluster.local

controlplane ~ → k exec test-nslookup -it -- nslookup nginx-resolver-service > /root/CKA/nginx.svc
Vi 2 pod cùng ns nên service name là đủ

controlplane ~ → cat /root/CKA/nginx.svc
Server:      10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      nginx-resolver-service
Address 1: 10.100.159.90 nginx-resolver-service.default.svc.cluster.local

controlplane ~ → k exec test-nslookup -it -- nslookup 10-244-192-1.default.pod.cluster.local
Server:      10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      10-244-192-1.default.pod.cluster.local
Address 1: 10.244.192.1 10-244-192-1.nginx-resolver-service.default.svc.cluster.local

controlplane ~ → k exec test-nslookup -it -- nslookup 10-244-192-1.default.pod.cluster.local > /root/CKA/nginx.pod
controlplane ~ →
```

Search Results

About 4,790 results (0.22 seconds)

DNS for Services and Pods | Kubernetes

Kubernetes > docs > concepts > services-networking > dns-pod-service

11 Apr 2022 ... Kubernetes **DNS** schedules a **DNS** Pod and Service on the cluster, and configures the kubelets to tell individual containers to use the **DNS** ...

Xem cách nslookup cho Pod

Q7 - Answer

```
kubectl run nginx-resolver --image=nginx
kubectl expose pod nginx-resolver --name=nginx-resolver-service --port=80 --target-port=80
--type=ClusterIP
kubectl run test-nslookup --image=busybox:1.28 -- sleep 1d
k get pod -o wide
k exec test-nslookup -it -- nslookup nginx-resolver-service > /root/CKA/nginx.svc
k exec test-nslookup -it -- nslookup 10-244-192-1.default.pod.cluster.local > /root/CKA/nginx.pod
```

Q8



1 2 3 4 5 6 7 8

Weight: 15

Create a static pod on `node01` called `nginx-critical` with image `nginx` and make sure that it is recreated/restarted automatically in case of a failure.

Use `/etc/kubernetes/manifests` as the Static Pod path for example.

- static pod configured under `/etc/kubernetes/manifests` ?
- Pod `nginx-critical-node01` is up and running

Q8 - Answer

```
controlplane ~ → k get nodes
NAME      STATUS   ROLES    AGE   VERSION
controlplane Ready    control-plane 59m   v1.27.0
node01     Ready    <none>     59m   v1.27.0

controlplane ~ → ssh node01
Warning: Permanently added the ECDSA host key for IP address '192.20.201.2' to the list of known hosts.
```

```
root@node01 ~ → k get pod
-bash: k: command not found
```

Chạy lệnh kubectl ở node01 không đc nên phải chạy ở ngoài, rồi copy/paste

```
root@node01 ~ ✗ kubectl get po
E1210 10:07:47.833736 7730 memcache.go:265] couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused
E1210 10:07:47.834231 7730 memcache.go:265] couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused
E1210 10:07:47.835706 7730 memcache.go:265] couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused
E1210 10:07:47.837248 7730 memcache.go:265] couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused
E1210 10:07:47.838732 7730 memcache.go:265] couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused
The connection to the server localhost:8080 was refused - did you specify the right host or port?
```

```
root@node01 ~ ✗ kubectl run nginx-critical --image=nginx --dry-run=client -o yaml > static.yaml
E1210 10:08:47.003497 7846 memcache.go:265] couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused
The connection to the server localhost:8080 was refused - did you specify the right host or port?
```

```
root@node01 ~ ✗ cd /etc/kubernetes/manifests/
```

```
root@node01 /etc/kubernetes/manifests → ls
```

kubelet sẽ control static pod ở đây

```
root@node01 /etc/kubernetes/manifests → vi nginx-critical.yaml
```

```
controlplane ~ → k get po
NAME                                READY   STATUS    RESTARTS   AGE
nginx-critical-node01              1/1     Running   0           8s
nginx-resolver                     1/1     Running   0          16m
test-nslookup                      1/1     Running   0          16m
```

```
kubectl run nginx-critical --image=nginx --dry-run=client -o yaml > static.yaml
```

```
cat static.yaml #Copy content
```

```
cd /etc/kubernetes/manifests/
```

```
vi nginx-critical.yaml #Paste content
```